

Name: Prajakta Mahangare Class: CSE(AI) SEDA Batch:B2 Rollno. 34 PRN: 12520047

Assignment No2. Perform the following operations using Python on the data sets: Compute and display summary statistics for each feature available in the dataset. (e.g. minimum value, maximum value, mean, range, standard deviation, variance and percentiles) Illustrate the feature distributions using histogram. Data cleaning, Data integration, Data transformation, Data model building (e.g. Classification)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('diabetes.csv')
print(df.head())
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Shape

```
df.shape
```

(768, 9)

```
df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
Column Non-Null Count Dtype
--- -
0 Pregnancies 768 non-null int64
1 Glucose 768 non-null int64
2 BloodPressure 768 non-null int64
3 SkinThickness 768 non-null int64
4 Insulin 768 non-null int64
5 BMI 768 non-null float64
6 DiabetesPedigreeFunction 768 non-null float64
7 Age 768 non-null int64
8 Outcome 768 non-null int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000	

Data Cleaning

Checking Missing Values

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Handle Zero Values

```
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols:
    df[col] = df[col].replace(0, df[col].mean())
```

Remove Duplicate Records

```
df.drop_duplicates(inplace=True)
```

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.681605	72.254807	26.606479	118.660163	32.450805	0.471876	33.240885	0.348958
std	3.369578	30.436016	12.115932	9.631241	93.080358	6.875374	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	20.536458	79.799479	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	79.799479	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

```
df['Glucose'].min(),df['Glucose'].max()
```

(44.0, 199.0)

```
for column in df.columns:
    print(f"\nFeature: {column}")
    print("Minimum:", df[column].min())
    print("Maximum:", df[column].max())
    print("Mean:", df[column].mean())
    print("Median:", df[column].median())
    print("Mode:", df[column].mode()[0])
    print("Range:", df[column].max() - df[column].min())
    print("Standard Deviation:", df[column].std())
    print("Variance:", df[column].var())
    print("25th Percentile:", df[column].quantile(0.25))
    print("50th Percentile:", df[column].quantile(0.50))
    print("75th Percentile:", df[column].quantile(0.75))
```

Range: 832.0
Standard Deviation: 93.08035765165374
Variance: 8663.952980559774
25th Percentile: 79.79947916666667
50th Percentile: 79.79947916666667
75th Percentile: 127.25

Feature: BMI
Minimum: 18.2
Maximum: 67.1
Mean: 32.4508051554362
Median: 32.0
Mode: 32.0
Range: 48.89999999999999
Standard Deviation: 6.875373507418385
Variance: 47.27076086651059
25th Percentile: 27.5
50th Percentile: 32.0
75th Percentile: 36.6

Feature: DiabetesPedigreeFunction
Minimum: 0.078
Maximum: 2.42
Mean: 0.47187630208333325
Median: 0.3725
Mode: 0.254
Range: 2.342
Standard Deviation: 0.33132859501277484
Variance: 0.10977863787313938
25th Percentile: 0.24375
50th Percentile: 0.3725
75th Percentile: 0.62625

Feature: Age
Minimum: 21
Maximum: 81
Mean: 33.240885416666664
Median: 29.0
Mode: 22
Range: 60
Standard Deviation: 11.76023154067868
Variance: 138.30304589037365
25th Percentile: 24.0
50th Percentile: 29.0
75th Percentile: 41.0

Feature: Outcome
Minimum: 0
Maximum: 1
Mean: 0.3489583333333333
Median: 0.0
Mode: 0
Range: 1
Standard Deviation: 0.4769513772427971
Variance: 0.22748261625380098
25th Percentile: 0.0
50th Percentile: 0.0
75th Percentile: 1.0

Mean

```
df['Glucose'].mean()
```

```
np.float64(121.68160502115886)
```

Median

```
df['Glucose'].median()

117.0
```

Mode

```
df['Glucose'].mode()
```

	Glucose
0	99.0
1	100.0

dtype: float64

Range

```
df['Glucose'].max() - df['Glucose'].min()
```

155.0

```
df['Glucose'].var(), df['Glucose'].std()
```

(926.3510482755512, 30.436015643897136)

Variance

```
df['Glucose'].var()
```

926.3510482755512

Standard Deviation

```
df['Glucose'].std()
```

30.436015643897136

Percentiles/Quantiles

```
df['Glucose'].quantile(0.25)
df['Glucose'].quantile(0.50)
df['Glucose'].quantile(0.75)
```

np.float64(140.25)

Count

```
df['Glucose'].count()
```

np.int64(768)

Value Counts

```
df['Outcome'].value_counts()
```

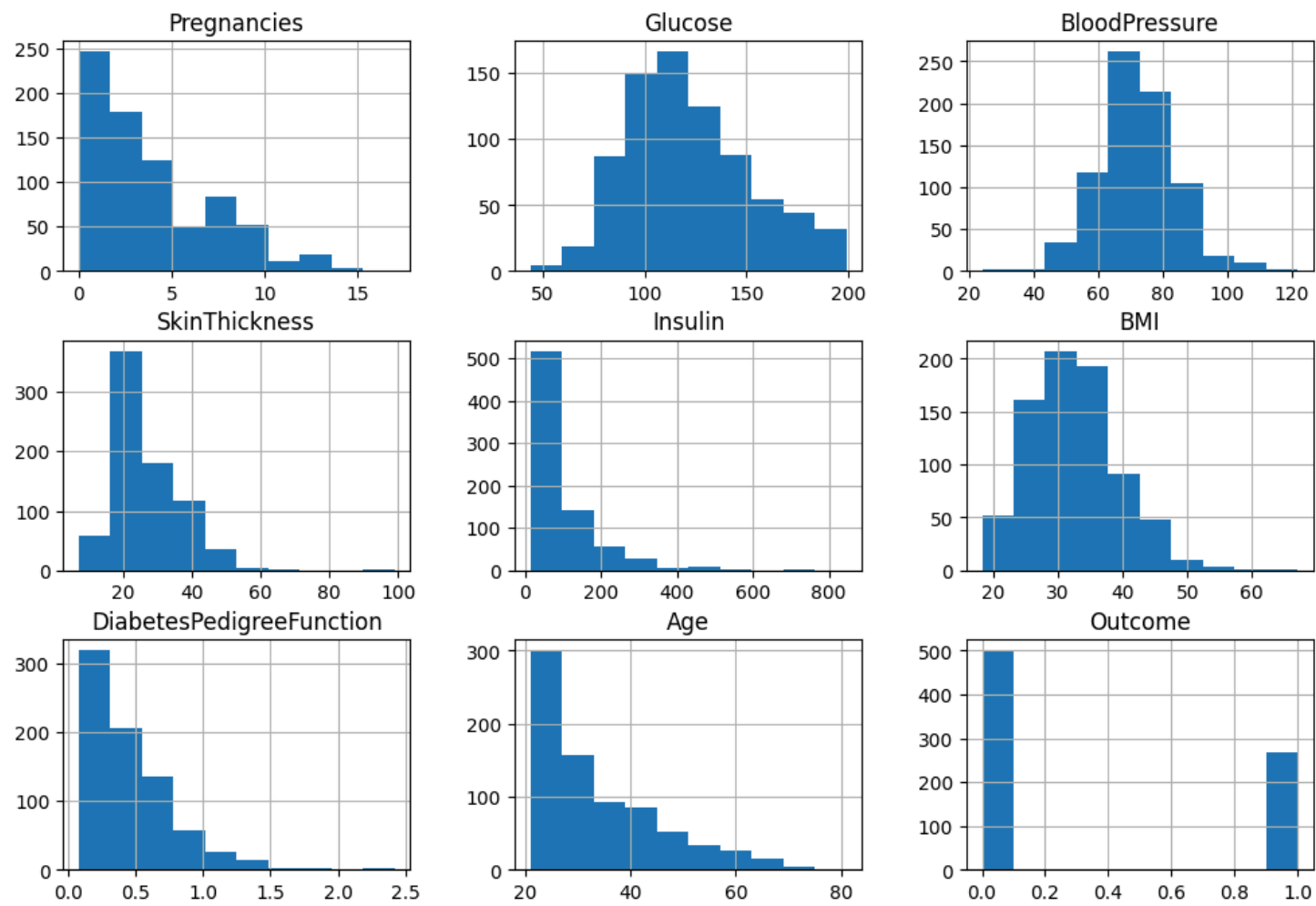
	count
Outcome	
0	500
1	268

dtype: int64

FEATURE DISTRIBUTION (HISTOGRAM)

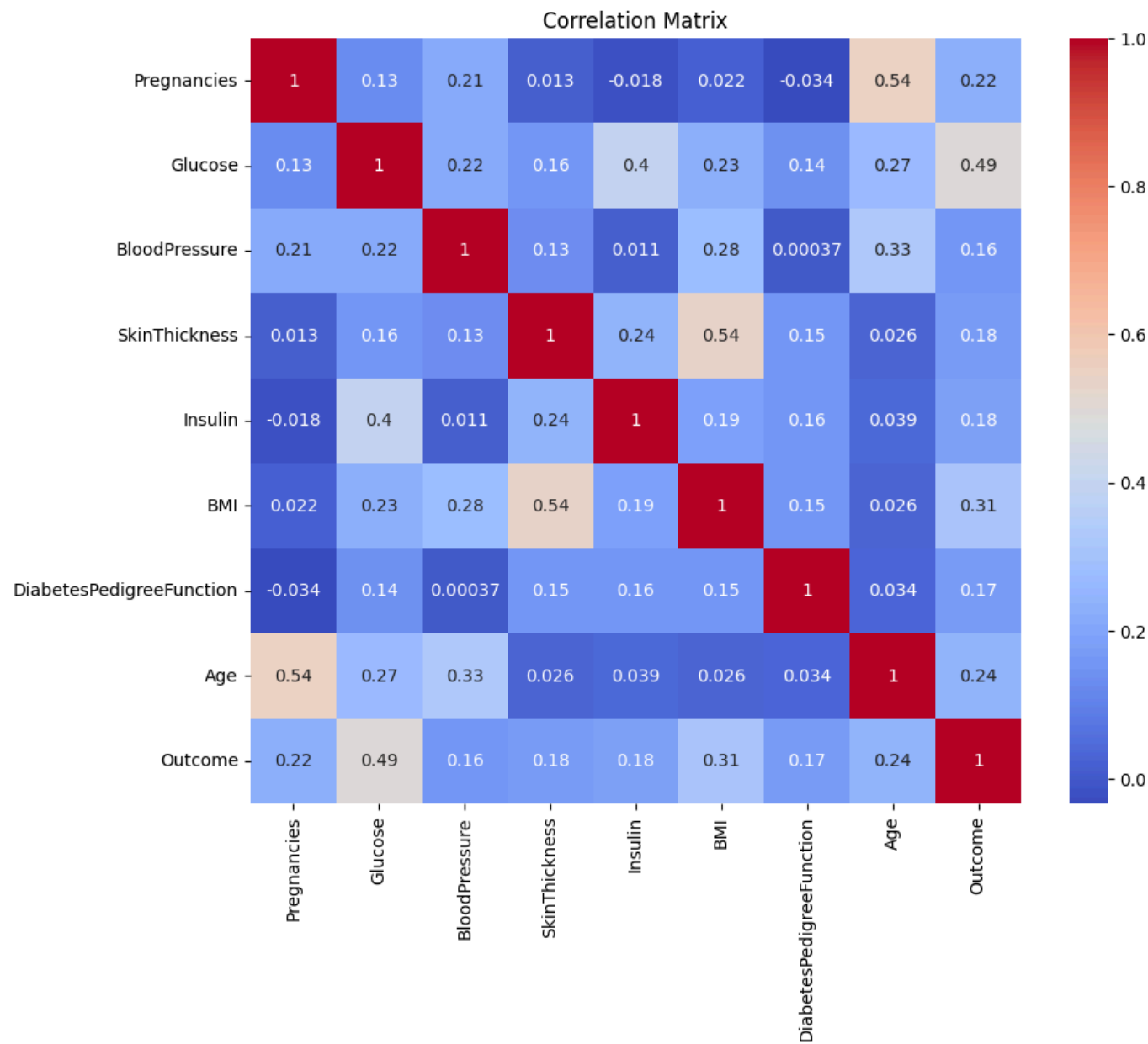
```
df.hist(figsize=(12,8), bins=10)
plt.suptitle("Feature Distribution using Histogram")
plt.show()
```

Feature Distribution using Histogram



Correlation Heatmap

```
import seaborn as sns
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Data Transformation

```
from sklearn.preprocessing import StandardScaler

X = df.drop("Outcome", axis=1)
y = df["Outcome"]

X_normalized = (X - X.min()) / (X.max() - X.min())
```

Data Model Buildinnng

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_normalized, y, test_size=0.25, random_state=0)
```

```
model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
y_pred = model.predict(X_test)
accuracy_score(y_test, y_pred)
```

0.78125

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Accuracy: 0.78125

Confusion Matrix:

```
[[118  12]
 [ 30  32]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.91	0.85	130
1	0.73	0.52	0.60	62
accuracy			0.78	192
macro avg	0.76	0.71	0.73	192
weighted avg	0.77	0.78	0.77	192

```
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

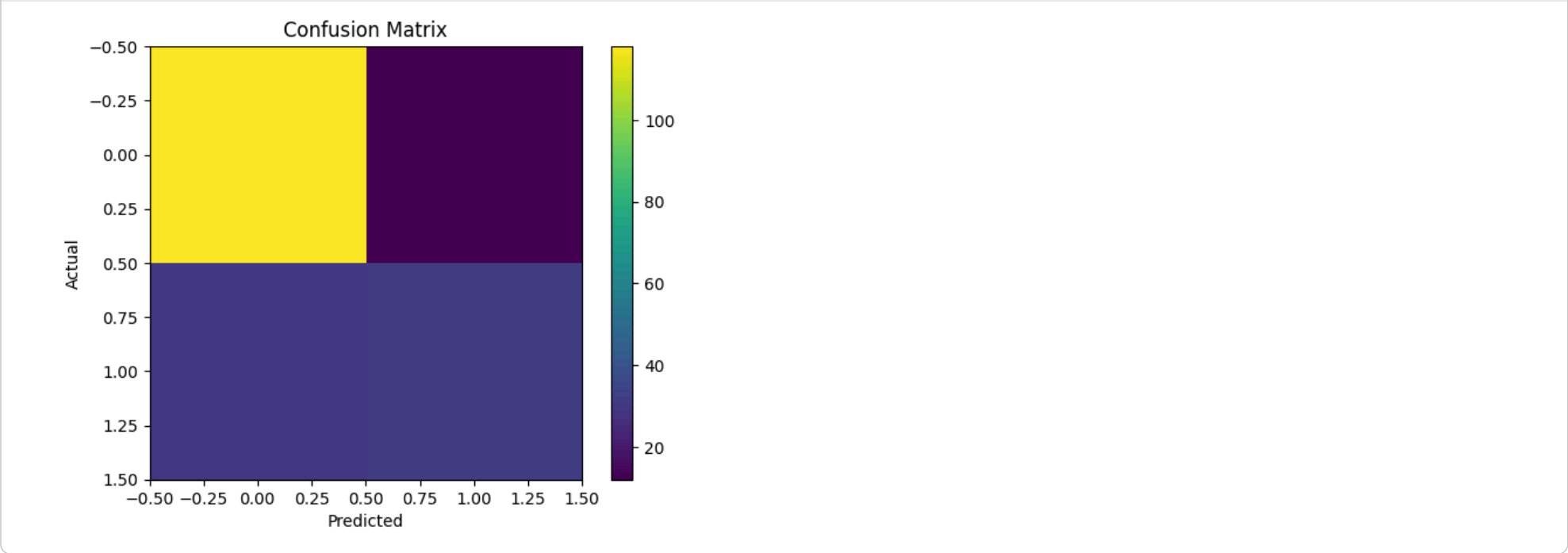
print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

Mean Squared Error: 0.21875
R2 Score: -0.00049627779156325592

```
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.imshow(cm)
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



User Defined Loop-Based Statistics

```
import math

def calculate_statistics(column_data):

    n = len(column_data)

    # Mean
    total = 0
    for value in column_data:
        total += value
    mean = total / n

    # Median
    sorted_data = sorted(column_data)
    if n % 2 == 0:
        median = (sorted_data[n//2 - 1] + sorted_data[n//2]) / 2
    else:
        median = sorted_data[n//2]

    # Mode
    freq = {}
    for value in column_data:
        freq[value] = freq.get(value, 0) + 1
    mode = max(freq, key=freq.get)

    # Variance
    variance_sum = 0
    for value in column_data:
        variance_sum += (value - mean) ** 2
    variance = variance_sum / n

    # Standard Deviation
    std_dev = math.sqrt(variance)

    # Range
    range_value = max(column_data) - min(column_data)

    return mean, median, mode, variance, std_dev, range_value
```

```
for column in df.columns:
    print("\nFeature:", column)

    data = list(df[column])

    mean, median, mode, variance, std_dev, range_value = calculate_statistics(data)

    print("Mean:", mean)
    print("Median:", median)
    print("Mode:", mode)
    print("Variance:", variance)
    print("Standard Deviation:", std_dev)
    print("Range:", range_value)
```

Standard Deviation: 30.416194075110315
Range: 155.0

Feature: BloodPressure
Mean: 72.25480651855469
Median: 72.0
Mode: 70.0
Variance: 146.6046581309444
Standard Deviation: 12.108041052579248
Range: 98.0

Feature: SkinThickness
Mean: 26.606479220920118
Median: 23.0
Mode: 20.536458333333332
Variance: 92.64001534046764
Standard Deviation: 9.624968329322837
Range: 92.0

Feature: Insulin
Mean: 118.66016303168442
Median: 79.79947916666667
Mode: 79.79947916666667
Variance: 8652.671791783003
Standard Deviation: 93.01973872132196
Range: 832.0

Feature: BMI
Mean: 32.45080515543617
Median: 32.0
Mode: 32.0
Variance: 47.20921039663232
Standard Deviation: 6.870895894760182
Range: 48.89999999999999

Feature: DiabetesPedigreeFunction
Mean: 0.4718763020833327
Median: 0.3725
Mode: 0.254


```
Median: 0.0
Mode: 0
Variance: 0.22718641493055383
Standard Deviation: 0.4766407608782046
Range: 1
```

User Defined Statistics Class

```
class Statistics:

    def __init__(self, data):
        self.data = sorted(data)
        self.n = len(data)

    def mean(self):
        return sum(self.data) / self.n

    def median(self):
        if self.n % 2 == 0:
            return (self.data[self.n//2 - 1] + self.data[self.n//2]) / 2
        else:
            return self.data[self.n//2]

    def variance(self):
        mean = self.mean()
        return sum((x - mean)**2 for x in self.data) / self.n

    def std_dev(self):
        return math.sqrt(self.variance())

    def data_range(self):
        return max(self.data) - min(self.data)
```

Use Statistics Class

```
for column in df.columns:
    print("\nUsing Class for:", column)

    stats = Statistics(list(df[column]))

    print("Mean:", stats.mean())
    print("Median:", stats.median())
    print("Variance:", stats.variance())
    print("Standard Deviation:", stats.std_dev())
    print("Range:", stats.data_range())
```

```
Standard Deviation: 3.3673836124089958
Range: 17

Using Class for: Glucose
Mean: 121.68160502115886
Median: 117.0
Variance: 925.1448620147744
Standard Deviation: 30.41619407511029
Range: 155.0

Using Class for: BloodPressure
Mean: 72.25480651855469
Median: 72.0
Variance: 146.6046581309444
Standard Deviation: 12.108041052579248
Range: 98.0

Using Class for: SkinThickness
Mean: 26.60647922092014
Median: 23.0
Variance: 92.64001534046766
Standard Deviation: 9.624968329322838
Range: 92.0

Using Class for: Insulin
Mean: 118.66016303168402
Median: 79.79947916666667
Variance: 8652.67179178304
Standard Deviation: 93.01973872132216
Range: 832.0

Using Class for: BMI
Mean: 32.4508051554362
Median: 32.0
Variance: 47.209210396632294
Standard Deviation: 6.87089589476018
Range: 48.89999999999999

Using Class for: DiabetesPedigreeFunction
Mean: 0.4718763020833337
Median: 0.3725
Variance: 0.10963569693840874
Standard Deviation: 0.331112816028629
Range: 2.342

Using Class for: Age
Mean: 33.240885416666664
Median: 29.0
Variance: 138.12296379937067
Standard Deviation: 11.752572645994181
Range: 60

Using Class for: Outcome
Mean: 0.3489583333333333
Median: 0.0
Variance: 0.22718641493055558
Standard Deviation: 0.47664076087820645
Range: 1
```

